

SSD_I2C usage reference

1 Overview

SSD20x provides two sets of HW I2C, as shown in the following table: (For actual use, please refer to Hw_Checklist provided by hardware HW):

The configuration of the first group of HW I2C GROUP0 can be selected from one of the following four, and the corresponding node is /dev/i2c-0.

The configuration of the second group of HW I2C GROUP1 can be selected from one of the following five types, and the corresponding node is /dev/i2c-1.

BUS	DEV	MODE	SCL	SDA
HW I2C GROUP0	/dev/i2c-0	MODE1	PAD_HDMITX_SCL	PAD_HDMITX_SDA
		MODE2	PAD_TTL1	PAD_TTL2
		MODE3	PAD_TTL14	PAD_TTL15
		MODE4	PAD_GPIO6	PAD_GPIO7
HW I2C GROUP1	/dev/i2c-1	MODE1	PAD_GPIO2	PAD_GPIO3
		MODE2	PAD_HDMITX_SCL	PAD_HDMITX_SDA
		MODE3	PAD_FUART_CTS	PAD_FUART_RTS
		MODE4	PAD_TTL22	PAD_TTL23
		MODE5	PAD_SD_CLK	PAD_SD_CMD



2. Configuration Instructions

2.1. I2C configuration under Kernel

2.1.1. Configuring config

CONFIG_I2C (enabled by default)

CONFIG_MS_I2C (enabled by default)

The kernel driver adopts the general device tree configuration method, and the corresponding dts is modified according to the configuration of each project

2.1.2. Padmode configuration

Here is an example of HW I2C GROUP0 and MODE1 (please select the dtsi file actually used by the project for configuration)

1. Open the `kernel\arch\arm\boot\dts\infinity2m-ssc011a-s01a-display.dtsi` file, i2c-group is configured as i2c-0, and i2c-padmux is configured as 1, that is, MODE1 is selected.

```
i2c0@0{
    compatible = "sstar,i2c";
    reg = <0x1F223000 0x200>, <0x1F203c00 0x200>, <0x1F207000 0x200>;
    #address-cells = <1>;
    #size-cells = <0>;
    clocks = <&CLK_miic0>;
    i2c-group = <0>;

    /*
     * padmux: 1 -> PAD_HDMITX_SCL, PAD_HDMITX_SDA
     *          2 -> PAD_TTL1, PAD_TTL2
     *          3 -> PAD_TTL14, PAD_TTL15
     *          4 -> PAD_GPIO6, PAD_GPIO7
     */
    i2c-padmux = <1>;

    /*
     * speed: 0 -> HWI2C_HIGH(high speed: 400 KHz)
     *         1 -> HWI2C_NORMAL(normal speed: 300 KHz)
     *         2 -> HWI2C_SLOW(slow speed: 200 KHz)
     *         3 -> HWI2C_VSLOW(very slow: 100 KHz)
     *         4 -> HWI2C_USLOW(ultra slow: 50 KHz)
     *         5 -> HWI2C_UVSLOW(ultra-very slow: 25 KHz)
     */
    i2c-speed = <3>;
    i2c-en-dma = <0>; // 0: disable; 1: enable;
    status = "ok";
};
```

2. Add padmux setting, open `kernel\arch\arm\boot\dts\infinity2m-ssc010a-s01a-padmux-display.dtsi`, for SSD20X platform, select mode1 in i2c-0, the corresponding pin is PAD_HDMITX_SCL/PAD_HDMITX_SDA.

```
//<PAD_HSYNC_OUT >,
//<PAD_VSYNC_OUT >,
<PAD_HDMITX_SCL PINMUX_FOR_I2C0_MODE_1 MDRV_PUSE_I2C0_SCL >,
<PAD_HDMITX_SDA PINMUX_FOR_I2C0_MODE_1 MDRV_PUSE_I2C0_SDA >,
//<PAD_HDMITX_HPD >,
```

2.2. I2C configuration of Uboot

2.2.1. Padmode configuration

1. Open `boot\include\configs\infinity2m.h`

Configure CONFIG_SYS_I2C_0_PADMUX/CONFIG_SYS_I2C_1_PADMUX to select the mode setting of I2C-0/I2C-1, and the mode here can match the kernel.

```

383: /* I2C configuration */
384: #define CONFIG_CMD_I2C
385: #define CONFIG_SYS_I2C
386: #define CONFIG_I2C_MULTI_BUS
387: #define CONFIG_SYS_I2C_MSTAR
388: #define I2C_MS_DECLARATIONS2
389: #define CONFIG_SYS_I2C_MS_SPEED (300000)
390: #define CONFIG_SYS_I2C_MS_SLAVE (0x7F)
391: /*
392:  * padmux: 1 -> PAD_HDMITX_SCL, PAD_HDMITX_SDA
393:  *          2 -> PAD_TTL1, PAD_TTL2
394:  *          3 -> PAD_TTL14, PAD_TTL15
395:  *          4 -> PAD_GPIO6, PAD_GPIO7
396:  */
397: #define CONFIG_SYS_I2C_0_PADMUX (0x1)
398: /*
399:  * padmux: 1 -> PAD_GPIO2, PAD_GPIO3
400:  *          2 -> PAD_HDMITX_SCL, PAD_HDMITX_SDA
401:  *          3 -> PAD_FUART_CTS, PAD_FUART_RTS
402:  *          4 -> PAD_TTL22, PAD_TTL23
403:  *          5 -> PAD_SD_CLK, PAD_SD_CMD
404:  */
405: #define CONFIG_SYS_I2C_1_PADMUX (0x1)
406:

```

3. Instructions for use

The following is only an example of the basic interface, please refer to the demo for the detailed code part.

3.1. Read and write I2C under Kernel

1. Open device, FILE_NAME can be "/dev/i2c-0" or "/dev/i2c-1"

```

fd = open(FILE_NAME, O_RDWR);
if (!fd)
{
    printf("can not open file %s\n", FILE_NAME);
    return 0;
}

```

2. I2C read interface

```

42: static int i2c_read(int fd, unsigned char slave_addr, unsigned char reg_addr, unsigned char *value)
43: {
44:     unsigned char inbuf, outbuf;
45:     struct i2c_rdwr_ioctl_data packets;
46:     struct i2c_msg messages[2];
47:     /*
48:      * In order to read a register, we first do a "dummy write" by writing
49:      * 0 bytes to the register we want to read from. This is similar to
50:      * the packet in set_i2c_register, except it's 1 byte rather than 2.
51:      */
52:     inbuf = reg_addr;
53:     messages[0].addr = slave_addr;
54:     messages[0].flags = 0; //write
55:     messages[0].len = sizeof(inbuf);
56:     messages[0].buf = &inbuf;
57:     /* The data will get returned in this structure */
58:     messages[1].addr = slave_addr;
59:     messages[1].flags = I2C_M_RD/* | I2C_M_NOSTART*/; //read
60:     messages[1].len = sizeof(outbuf);
61:     messages[1].buf = &outbuf;
62:     /* Send the request to the kernel and get the result back */
63:     packets.msgs = messages;
64:     packets.nmsgs = 2;
65:     if(ioctl(fd, I2C_RDWR, &packets) < 0)
66:     {
67:         perror("Unable to send data");
68:         return 1;
69:     }
70:     *value = outbuf;
71:     return 0;
72: } « end i2c_read »

```

3. i2c write interface

```

static int i2c_write(int fd, unsigned char slave_addr, unsigned char reg_addr, unsigned char value)
{
    unsigned char inbuf[2];
    struct i2c_rdwr_ioctl_data packets;
    struct i2c_msg messages[1];
    messages[0].addr = slave_addr;
    messages[0].flags = 0;
    messages[0].len = sizeof(inbuf);
    messages[0].buf = inbuf;
    /* The first byte indicates which register we'll write */
    inbuf[0] = reg_addr;
    /*
    * The second byte indicates the value to write. Note that for many
    * devices, we can write multiple, sequential registers at once by
    * simply making outbuf bigger.
    */
    inbuf[1] = value;
    /* Transfer the i2c packets to the kernel and verify it worked */
    packets.msgs = messages;
    packets.nmsgs = 1;
    if(ioctl(fd, I2C_RDWR, &packets) < 0)
    {
        perror("Unable to send data");
        return 1;
    }
    return 0;
} « end i2c_write »

```

3.2. Read and write I2C under Uboot

The Uboot part has been connected to the standard interface.

reference document: `alkaid\boot\common\cmd_i2c.c`

```

SigmaStar # i2c
i2c - I2C sub-system

Usage:
i2c bus [muxtype:muxaddr:muxchannel] - show I2C bus info
crc32 chip address[.0, .1, .2] count - compute CRC32 checksum
i2c dev [dev] - show or set current I2C bus
i2c loop chip address[.0, .1, .2] [# of objects] - looping read of device
i2c md chip address[.0, .1, .2] [# of objects] - read from I2C device
i2c mm chip address[.0, .1, .2] - write to I2C device (auto-incrementing)
i2c mw chip address[.0, .1, .2] value [count] - write to I2C device (fill)
i2c nm chip address[.0, .1, .2] - write to I2C device (constant address)
i2c probe [address] - test for and show device(s) on the I2C bus
i2c read chip address[.0, .1, .2] length memaddress - read to memory
i2c write memaddress chip address[.0, .1, .2] length - write memory to i2c
i2c reset - re-init the I2C Controller
i2c speed [speed] - show or set I2C bus speed

```

A brief description of the test command:

1. Select bus, optional 0/1 bus: `i2c dev 1`
2. Detect devices on the corresponding bus: `i2c probe`
3. Read device data and print the value directly on the serial port: `i2c md slave_id reg_addr.len size`

```

// *slave_id represents the device id
// *reg_addr is the device register address to be read
// *len is the number of bytes of reg_addr (1/2 byte)
// *size is the length of the data to be read back

```

Example of touch IC using SSD20X public demo board:

The slave_id of the touch IC is generally 0x5D, the read version address reg_addr is 0x4081 (2 bytes), then `i2c md 0x5D 0x4081.2 0x2`

```

SigmaStar # i2c md 0x5d 0x4081.2 0x2
4081: 39 31
SigmaStar #

```